

CreatingAwatchOSApp 项目结构分析

CreatingAwatchOSApp 项目结构详细分析

一、项目整体架构

本项目是基于 SwiftUI 开发的跨平台地标应用，支持 **iOS**、**macOS**、**watchOS** 三大平台，采用“模块化 + 分层”设计，清晰分离数据模型、视图组件与平台特定功能。

项目版本划分

项目包含两个核心版本，满足不同学习与使用需求：

- **StartingPoint**: 学习起点版本，仅包含基础功能实现（如 iOS 端核心地标展示），适合入门理解；
- **Complete**: 完整版本，覆盖全平台功能（iOS/macOS/watchOS）与所有业务逻辑，适合实际参考。

主要模块目录结构

```
CreatingAwatchOSApp/  
├── Complete/           # 完整版本目录  
│   ├── Landmarks/     # iOS 主应用（核心功能，含地标浏览、收藏等）  
│   ├── MacLandmarks/  # macOS 应用（适配桌面端，支持菜单命令、分栏布局）  
│   └── WatchLandmarks/ # watchOS 应用（适配手表端，简化交互与布局）  
└── StartingPoint/      # 学习起点版本目录  
    └── Landmarks/      # 基础 iOS 应用（仅含核心地标展示与列表功能）
```

二、数据模型层

数据模型层是应用的“数据基础”，定义数据结构、管理数据存储与流转，核心文件与功能如下：

1. 核心数据模型文件

(1) Landmark.swift (地标数据结构)

定义地标的基础属性与行为，是应用的核心数据实体：

```
struct Landmark: Hashable, Codable, Identifiable {
    var id: Int          // 地标唯一ID (Identifiable 协议必需)
    var name: String     // 地标名称
    var park: String     // 所属公园
    var state: String    // 所属州/地区
    var description: String // 地标描述
    var isFavorite: Bool // 是否为用户收藏 (默认 false)
    var isFeatured: Bool // 是否为特色地标 (用于首页推荐)

    // 图片相关属性 (处理本地图片加载)
    private var imageName: String
    var image: Image {
        Image(imageName)
    }

    // 地理位置属性 (经纬度, 用于地图显示)
    private var coordinates: Coordinates
    var locationCoordinate: CLLocationCoordinate2D {
        CLLocationCoordinate2D(
            latitude: coordinates.latitude,
            longitude: coordinates.longitude
        )
    }
}

// 嵌套结构体: 存储经纬度
struct Coordinates: Hashable, Codable {
    var latitude: Double
    var longitude: Double
}

// 分类属性 (按类型分组, 如湖泊、山脉)
var category: Category
enum Category: String, CaseIterable, Codable {
    case lakes = "Lakes"
    case rivers = "Rivers"
    case mountains = "Mountains"
}
```

```
}  
}
```

(2) ModelData.swift (数据管理类)

应用的“数据中枢”，负责加载、存储与管理全局数据，支持响应式更新：

```
@Observable // SwiftUI 响应式标记，数据变化时自动通知视图  
class ModelData {  
    // 地标数据（从 JSON 文件加载）  
    var landmarks: [Landmark] = load("landmarkData.json")  
    // 徒步数据（从 JSON 文件加载）  
    var hikes: [Hike] = load("hikeData.json")  
    // 用户配置（默认值，支持后续修改）  
    var profile = Profile.default  
  
    // 计算属性：过滤特色地标（用于首页推荐）  
    var features: [Landmark] {  
        landmarks.filter { $0.isFeatured }  
    }  
  
    // 计算属性：按类别分组地标（用于列表分类展示）  
    var categories: [String: [Landmark]] {  
        Dictionary(  
            grouping: landmarks,  
            by: { $0.category.rawValue }  
        )  
    }  
}  
  
// 辅助函数：从 JSON 文件加载数据（泛型支持，适配不同数据模型）  
func load<T: Decodable>(_ filename: String) -> T {  
    let data: Data  
    guard let file = Bundle.main.url(forResource: filename, withExtension: nil) else {  
        fatalError("Couldn't find \"(filename)\" in main bundle.")  
    }  
    do {  
        data = try Data(contentsOf: file)  
    } catch {  
        fatalError("Couldn't load \"(filename)\" from main bundle:\n\"(error)\"")  
    }  
}
```

```
do {  
    let decoder = JSONDecoder()  
    return try decoder.decode(T.self, from: data)  
} catch {  
    fatalError("Couldn't parse \("\(filename) as \("\(T.self):\\n\\(error)")  
}  
}
```

2. 数据持久化方式

应用通过多种方式实现数据持久化，确保数据不丢失且跨会话可用：

- **JSON 文件**：存储初始地标与徒步数据（应用打包时内置，不可修改）；
- **@AppStorage**：存储用户偏好设置（如地图缩放级别、是否显示收藏项），数据保存在 `UserDefaults` 中；
- **@Environment**：将 `ModelData` 实例注入全局环境，实现跨视图数据共享，避免重复创建与数据不一致。

三、视图层结构

视图层是应用的“用户界面”，采用“组件化”设计，按功能与类型拆分视图，便于复用与维护。

1. 视图目录结构（以 Complete/Landmarks 为例）

```
Views/  
├── Badges/    # 徽章相关视图（如徒步成就徽章）  
├── Categories/ # 类别相关视图（如分类标题、分类列表行）  
├── Helpers/   # 辅助视图组件（如加载指示器、空数据提示）  
├── Hikes/     # 徒步相关视图（徒步详情、路线图、统计信息）  
├── Landmarks/ # 地标核心视图（列表、详情、地图、收藏按钮）  
├── PageView/  # 分页视图组件（用于首页特色地标轮播）  
└── Profiles/  # 用户资料相关视图（资料编辑、偏好设置）
```

2. 主要视图组件详解

(1) 应用入口与根视图

- **LandmarksApp.swift (应用入口) :**

用 `@main` 标记为应用入口，初始化 `ModelData` 并注入全局环境，适配不同平台场景：

```
@main
struct LandmarksApp: App {
    // 初始化全局数据模型
    @State private var modelData = ModelData()

    var body: some Scene {
        WindowGroup {
            // 根视图: ContentView, 注入 ModelData
            ContentView()
                .environment(modelData)
        }
        // macOS: 添加菜单命令
        #if !os(watchOS)
        .commands {
            LandmarkCommands()
        }
        #endif
        // macOS: 添加设置面板
        #if os(macOS)
        Settings {
            LandmarkSettings()
        }
        #endif
        // watchOS: 添加通知场景
        #if os(watchOS)
        WKNotificationScene(
            controller: NotificationController.self,
            category: "LandmarkNear"
        )
        #endif
    }
}
```

- **ContentView.swift (根视图) :**

应用的顶层视图，用 `TabView` 实现 “精选” 与 “列表” 两大功能标签页切换：

```
struct ContentView: View {
    @State private var selection: Tab = .featured // 当前选中的标签

    // 枚举：定义标签类型
    enum Tab {
        case featured // 精选标签（首页轮播+分类）
        case list // 列表标签（全地标列表+筛选）
    }

    var body: some View {
        TabView(selection: $selection) {
            // 精选标签页：CategoryHome
            CategoryHome()
                .tabItem {
                    Label("Featured", systemImage: "star")
                }
                .tag(Tab.featured)

            // 列表标签页：LandmarkList
            LandmarkList()
                .tabItem {
                    Label("List", systemImage: "list.bullet")
                }
                .tag(Tab.list)
        }
    }
}
```

（2）地标核心视图

◦ `CategoryHome.swift`（精选标签页）：

展示 “特色地标轮播” 与 “分类地标列表”，支持模态弹出用户资料：

```
struct CategoryHome: View {
    @Environment(ModelData.self) var modelData // 共享数据模型
    @State private var showingProfile = false // 控制资料弹窗显示
```

```

var body: some View {
    NavigationStack {
        ScrollView {
            // 特色地标轮播（PageView 组件）
            PageView(
                pages: modelData.features.map { FeatureCard(landmark: $0) }
            )
            .frame(height: 200)

            // 分类地标列表（按类别分组）
            ForEach(modelData.categories.keys.sorted(), id: \.self) { key in
                CategoryRow(
                    categoryName: key,
                    items: modelData.categories[key]!
                )
            }
            .padding(.horizontal)
        }
        .navigationTitle("Featured")
        // 右上角按钮：弹出用户资料
        .toolbar {
            Button {
                showingProfile.toggle()
            } label: {
                Label("User Profile", systemImage: "person.crop.circle")
            }
        }
        // 模态弹窗：用户资料页
        .sheet(isPresented: $showingProfile) {
            ProfileHost()
                .environment(modelData)
        }
    }
}
}

```

◦ **LandmarkList.swift（地标列表页）：**

展示全地标列表，支持“按类别筛选”“仅显示收藏”，适配大屏设备的分栏布局：

```

struct LandmarkList: View {
    @Environment(ModelData.self) var modelData // 共享数据模型
    @State private var showFavoritesOnly = false // 筛选：仅显示收藏
    @State private var selectedCategory: Landmark.Category? // 筛选：按类别

    // 过滤后的地标列表（根据筛选条件动态计算）
    var filteredLandmarks: [Landmark] {
        modelData.landmarks.filter { landmark in
            (!showFavoritesOnly || landmark.isFavorite) &&
            (selectedCategory == nil || landmark.category == selectedCategory)
        }
    }

    var body: some View {
        // 分栏布局：适配 iPad/macOS（左侧列表，右侧详情）
        NavigationSplitView {
            List(selection: $selectedCategory) {
                // 筛选开关：仅显示收藏
                Toggle(isOn: $showFavoritesOnly) {
                    Label("Favorites Only", systemImage: "star.fill")
                }

                // 按类别分组的列表（可点击类别筛选）
                ForEach(Landmark.Category.allCases) { category in
                    Section(header: Text(category.rawValue)) {
                        ForEach(filteredLandmarks.filter { $0.category == category }) { landmark in
                            NavigationLink {
                                LandmarkDetail(landmark: landmark)
                            } label: {
                                LandmarkRow(landmark: landmark)
                            }
                        }
                    }
                }
            }
            .navigationTitle("Landmarks")
            .listStyle(.sidebar) // 侧边栏样式（适配分栏）
        } detail: {
            // 详情区域：选中地标时显示详情，否则显示提示
            if let selectedCategory {
                CategoryHome(selectedCategory: selectedCategory)
            }
        }
    }
}

```



```

    } else {
        Text("Select a Landmark")
    }
}
}
}
}

```

- **LandmarkDetail.swift（地标详情页）：**

展示地标的详细信息（图片、描述、地图位置），支持切换收藏状态：

```

struct LandmarkDetail: View {
    @Environment(ModelData.self) var modelData // 共享数据模型
    let landmark: Landmark // 当前选中的地标（从列表传递）

    // 计算当前地标在数据模型中的索引（用于修改收藏状态）
    var landmarkIndex: Int {
        modelData.landmarks.firstIndex(where: { $0.id == landmark.id })!
    }

    var body: some View {
        ScrollView {
            // 顶部大图
            MapView(coordinate: landmark.locationCoordinate)
                .frame(height: 300)

            // 圆形图标（叠加在地图上方，偏移显示）
            CircleImage(image: landmark.image)
                .offset(y: -130)
                .padding(.bottom, -130)

            // 地标信息（名称、公园、州）
            VStack(alignment: .leading) {
                HStack {
                    Text(landmark.name)
                        .font(.title)
                    // 收藏按钮：点击切换收藏状态
                    FavoriteButton(isSet: $modelData.landmarks[landmarkIndex].isFavorite)
                }

                HStack {

```

```

        Text(landmark.park)
        Spacer()
        Text(landmark.state)
    }
    .font(.subheadline)
    .foregroundColor(.gray)

    Divider()

    // 地标描述
    Text("About \${landmark.name}")
        .font(.title2)
    Text(landmark.description)
}
.padding()
}
.navigationTitle(landmark.name)
.navigationBarTitleDisplayMode(.inline)
}
}

```

(3) 辅助视图组件

- **MapView.swift (地图视图) :**

集成 MapKit 显示地标位置，支持保存用户的地图缩放偏好：

```

struct MapView: View {
    var coordinate: CLLocationCoordinate2D // 地标经纬度
    // 地图缩放级别 (@AppStorage 持久化，用户修改后保存)
    @AppStorage("MapView.zoom") private var zoom: Zoom = .medium

    // 枚举：地图缩放级别
    enum Zoom: String, CaseIterable, Identifiable {
        case near = "Near"
        case medium = "Medium"
        case far = "Far"
        var id: Self { self }
    }
}

```

```

// 根据缩放级别计算区域半径
var zoomLevel: Double {
    switch zoom {
    case .near: return 200
    case .medium: return 500
    case .far: return 1000
    }
}

var body: some View {
    Map(initialPosition: .region(region)) {
        Marker("", coordinate: coordinate)
    }
    // 工具栏：添加缩放选择器
    .toolbar {
        ToolbarItem(placement: .bottomBar) {
            Picker("Zoom", selection: $zoom) {
                ForEach(Zoom.allCases) { level in
                    Text(level.rawValue)
                }
            }
            .pickerStyle(.segmented)
        }
    }
}

// 地图区域（根据经纬度和缩放级别计算）
private var region: MKCoordinateRegion {
    MKCoordinateRegion(
        center: coordinate,
        span: MKCoordinateSpan(latitudeDelta: zoomLevel / 100000, longitudeDelta: zoomLevel
/ 100000)
    )
}
}

```

◦ FavoriteButton.swift（收藏按钮）：

自定义按钮组件，支持切换地标收藏状态，带点击动画：

```
struct FavoriteButton: View {
    @Binding var isSet: Bool // 双向绑定：收藏状态（true=已收藏）

    var body: some View {
        Button(action: {
            isSet.toggle() // 点击切换状态
        }) label: {
            Label("Toggle Favorite", systemImage: isSet ? "star.fill" : "star")
                .labelStyle(.iconOnly)
                .foregroundColor(isSet ? .yellow : .gray)
                .animation(.easeInOut, value: isSet) // 状态变化动画
        }
    }
}
```

四、代码流程结构

1. 应用启动流程

1. **初始化入口**：LandmarksApp 被 @main 标记为应用入口，启动时初始化；
2. **数据加载**：创建 ModelData 实例，通过 load() 函数从 JSON 文件加载地标与徒步数据；
3. **视图构建**：创建 ContentView 作为根视图，通过 environment(modelData) 将数据模型注入全局环境；
4. **平台适配**：根据运行平台（iOS/macOS/watchOS）添加特定功能（如 macOS 菜单、watchOS 通知场景）。

2. 数据流向（单向数据流）

应用遵循 SwiftUI 单向数据流原则，确保数据一致与可追溯：

1. **自上而下传递**：ModelData 从应用顶层注入全局环境，子视图通过 @Environment(ModelData.self) 获取

（注：文档部分内容可能由 AI 生成）